VITyarthi | Open-Source Software

The Open-Source Audit

A Capstone Project for OSS NGMC Course

Course: Open-Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
Kaustubh Upadhyay	24BAI10060
Slot	**Date of Submission**
B22	29/March/2026

1. Introduction

Most of the software that powers the modern world — the servers that run websites, the tools that developers use every day, the operating systems inside supercomputers and smartphones — was not built by a single company and sold for profit. It was built openly, shared freely, and improved collectively. That is the essence of open source.

This project asks you to stop and think about that.

You will choose one real open-source software project — something you can download, read about, and interact with on Linux — and conduct a structured audit of it. Not just 'what does it do,' but why it exists, who built it, what freedoms its license actually gives you, what philosophical values it represents, and how it has shaped the world around it.

Alongside the written report, you will write five shell scripts that demonstrate practical Linux skills. These are not meant to be impressive programs — they are meant to show that you understand how open-source tools work at the command line, and how automation connects to the philosophy of sharing and transparency.

Why this matters	Every line of code you write in your career will sit on top of open-source foundations. Understanding the philosophy behind that is not optional — it is part of being a professional.

2. Choosing Your Software

Approved software options

Software	Category	License	Why it's interesting
Linux Kernel	Operating System	`GPL v2`	*The foundation everything else runs on*
Apache HTTP Server	Web Server	`Apache 2.0`	*Powers ~30% of all websites globally*
MySQL	Database	`GPL v2 / Commercial`	*A license with a dual-model story to tell*
Firefox	Web Browser	`MPL 2.0`	*A nonprofit fighting for an open web*
VLC Media Player	Multimedia	`LGPL / GPL`	*Plays anything — built by students in Paris*
LibreOffice	Office Suite	`MPL 2.0`	*Born from a community fork — a real lesson*
Python	Programming Language	`PSF License`	*A language shaped entirely by community*
Git	Version Control	`GPL v2`	*The tool Linus built when proprietary failed him*

Your choice	Write your chosen software here before submission: Git

3. Project Report

Part A — Origin and Philosophy (Units 1 & 2)

A1. The problem this software was created to solve

Before the development of Git, large scale software projects, for example, Linux Kernel, were struggling for collaboration as the biggest issue wasn't writing the code, it was managing it.

In the early 2000s, the development of Linux relied on a proprietary version control system called BitKeeper. Although it worked efficiently, it came with restrictions that went against the open-source philosophy. Eventually, sue to licencing disagreements, access to Bitkeeper was revoked. This left the Linux community without a reliable system to manage contributions.

This is where Linus Tovalds steeped in again. Instead of searching for another tool, he made a bold decision: build one from scratch.

Git was developed in 2005, not as polished product, but as a necessity. It was designed with very clear priorities:

1. **Speed:** It needed to handle large codebases like Linux efficiently.
2. **Distributed nature:** Every developer should have a full copy of the repository.
3. **Data integrity:** Changes should never be lost or corrupted.
4. **Flexibility:** It should adapt different workflows.

Unlike older systems, Git did not rely on a central server. Every developer had their own complete repository. This was revolutionary because it removed dependency on a single point of failure.

Git's creation wasn't just technical, it was philosophical. It reflected a belief that control should be decentralised and that developers should not depend on proprietary system to collaborate.

In many ways, Git solved not just a technical problem, but a power imbalance. It gave developers independence.

A2. The license — what it actually says

Git is licenced under the GNU General Public Licence v2(GPLv2). This license is one of the most influential in open-source history.

At its core, the GPL ensure that software remains free, not just in cost, but also in freedm.

The Four Freedoms of Free Software

1. Freedom to run the program for any purpose.
2. Freedom to study how the program works.
3. Freedom to modify the program.
4. Freedom to distribute copies.

Can companies modify and seel Git without sharing changes?

The GPL is copyleft licence, which means if one modifies Git and distributes it, they must also share their source code under the same licence. This ensures that improvement remain accessible to the community.

GPL vs MIT License

The difference between GPL and MIT licencing is that GPL forces openness, i.e. work must remain open whereas MIT is permissive, i.e. allows proprietary use without sharing changes. GPL is used when the requirement is to protect community contribution whereas MIT is used when maximum adoption is required even commercially.

Free as in Freedom vs Free as in Free Beer

Git's use of GPL shows that Git is free in money, "free beer", as well as control, "freedom".

Example, GitHub uses Git internally, but Git itself remains open and accessible to everyone.

A3. The ethics of open source

Open- source is not just about code, it's about values.

Should all software be open-source?

1. Argument for:
   a. Transparency improves security.
   b. Collaboration accelerates innovation.
   c. Knowledge becomes accessible globally.
2. Arguments against:
   a. Some software requires protection.
   b. Businesses need revenue models.
   c. Open source may lack structured accountability.

The truth lies somewhere in between. Not all software must be open, but critical infrastructure benefits from openness.

Is it ethical for companies to profit from open-source?

This is a complicated issue.

1. Red Hat
2. Google
3. Meta

All use open-source software extensively.

Is it exploitation? It depends.

1. If they contribute back=ethical.
2. If they only extract value=questionable.

Git itself is used by billion-dollar companies, yet its core remains community driven. That balance is that what makes open-source sustainable.

Standing on the shoulders of giants

This phares means building upon existing knowledge.

In software:

1. No one writes everything from scratch.
2. Every tool builds on previous tools.

Git itself is built on ideas from earlier versions control sytems.

Open source enables innovation because it removes repetition. Instead of reinventive the wheel, developers improve it.

Rather than killing originality, it accelerates it.

Part B — Linux Footprint (Unit 2)

Git's presence on a Linux system is subtle but powerful. Unlike server based software, Git operates quietly, yet it is deeply integrated into the development workflow.

Installation method —

Git is available through package managers:

1. Debian/Ubuntu: apt install git
2. Red Hat/Fedora: yum install git or dnf install git

It can also be compiled from source for customization.

This reflects open-source flexibility, i.e., not locked into one method.

Key directories —

Once installed, Git spreads across several directories:

1. /user/bin/git=main executable
2. /etc/gitconfig=system wide configuration
3. ~/.gitcongig=user specific configuration
4. .git/(inside projects) =repository metadata

The .git folder is especially important. It contains:

1. Commit history
2. Branch data
3. Configuration

The Git philosophy is: Everything is transparent and accessible.

User and group —

Git runs as the current user, not a dedicated system user.

This is important for security:

1. No unnecessary privileges.
2. Minimal risk of system wide damage.

However, it shared environments, Git repositories may use group permissions to allow collaboration.

Service management —

Git is not a background service. It runs on demand.

However, when used with servers(like GitHub or GitLab Enterprise), it integrates with services:

1. systemctl start gitlab
2. systemctl stop gitlab

This separation keeps Git lightweight.

Update model —

Git updates come from:

1. Linux package repositories
2. Source code releases

Security patches are:

1. Reviewed by the community
2. Distributed openly

Unlike proprietary systems, you don't have to trust a company blindly, you can verify changes yourself.

Why Git's Linux Footprint Matters?

Git reflects Unix philosophy:

1. Do one thing well
2. Stay simple
3. Be composable

Part C — The FOSS Ecosystem (Units 3 & 4)

Git does not exist alone. It is part of a vast ecosystem that supports modern software development.

Developments

Fit relies on several open-source components:

1. C standard libraries
2. Shell environment
3. SSH for remote communication
4. Open SSL for secure data transfer

Open source is collaborative at every layer.

What has it enabled or inspired?

Git has transformed how software is built.

It has enabled:

1. GitHub and GitLab platforms
2. Open-source collaboration at scale
3. Distributed team across the world

Without Git, platforms like GitHub wouldn't exist in their current form

It has inspired:

1. For based development
2. Pull based workflows
3. Continuous integration systems

Where does it fit in the LAMP stack, or how does it relate to the web?

Git is not the part of the LAMP stack directly, bit indirectly supports it.

LAMP=Linux+Apache+MySQL+PHP

Git helps developers:

1. Manage web application code
2. Track changes
3. Deploy updates

It acts as the backbone of development pipelines.

Is there a community—mailing lists, forums, conferences, IRC channels? Who governs it?

Git has a strong, decentralised community.

1. Maintained by core contributors
2. Contributions reviewed publicly
3. Discussions happen via mailing lists

Unlike corporate controlled software, Git evolves through consensus and merit.

Why does the Ecosystem matters?

Git's access is not just because of its design, it's because of its ecosystem.

It thrives because:

1. Developers trust it
2. Communities support it
3. Tools integrate with it

Part D — Open-Source vs Proprietary (Critical Analysis)

Dimension	Open Source (Git)	Proprietary Alternative (Perforce Helix Core)
Cost	Complete free to use, modify and distribute.	Requires paid licencing for it features.
Security (who can audit the code?)	Code is open and can be audited by anyone.	Source code is closed, user must trust the vendor.
Support and reliability	Community driven support, forums and documentation.	Dedicated enterprise support as SLA's.

Freedom to modify	Full freedom to modify and redistributed (GPL).	No access to source code, no modification allowed.
Community vs corporate control	High scalable for distributed say yeast.	Optimised for large centralised workflows.
Your overall verdict	Ideal for Open-source development, small teams, distributed collaboration and developers who value freedom and control.	Ideal for large enterprises, teams working with massive binary file, environments requiring strict control and support.

If we had to choose for real-world deployment, I would recommend Git in most cases. Its flexibility, zero cost and strong community make it a practical and sustainable choice.

However, we also recognize that in specialized environment, like large-scale game development, Perforce may be more efficient.

Will I contribute to Git? Yes, as contributing to Git is not just about improving a tool, it is about participating in a global ecosystem built on collaboration, trust and shared knowledge.

Git represents more than software. It represents a way of thinking that progress is faster when it is shared.

4. Shell Script Tasks

Script 1 — System Identity Report (Unit 1 & 2)

Write a script that introduces the Linux system like a welcome screen. It should display:

- The Linux distribution name and kernel version
- The current logged-in user and their home directory
- System uptime and current date/time
- A message stating which open-source license covers the OS

Concepts to use: variables, echo, command substitution ($()), basic output formatting.

Starter structure

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Kaustubh Upadhyay
# Course: Open Source Software

# --- Variables
STUDENT_NAME="Kaustubh Upadhyay"
SOFTWARE_CHOICE="Git"

# --- System info using command substitution
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DISTRO=$(lsb_release -ds 2>/dev/null || echo "Linux Distribution")
CURRENT_DATE=$(date "+%A, %d %B %Y")

# --- Display
echo "=========================================="
echo "          Open Source Audit               "
echo "=========================================="
echo "Student Name    : $STUDENT_NAME"
echo "Chosen Software : $SOFTWARE_CHOICE"
echo "------------------------------------------"
echo "OS Distribution : $DISTRO"
echo "Kernel Version  : $KERNEL"
echo "Current User    : $USER_NAME"
echo "Home Directory  : $HOME"
echo "System Uptime   : $UPTIME"
echo "Current Date    : $CURRENT_DATE"
echo "------------------------------------------"
echo "License Note: This system and Git are both"
echo "covered under the GNU GPL v2 license."
echo "=========================================="
```

```
$ bash "C:\Users\kstbh\OneDrive\Desktop\script1.sh"
C:\Users\kstbh\OneDrive\Desktop\script1.sh: line 13: uptime: command not found
==========================================
          Open Source Audit
==========================================
Student Name     : Kaustubh Upadhyay
Chosen Software  : Git
------------------------------------------
OS Distribution  : Linux Distribution
Kernel Version   : 3.6.6-1cdd4371.x86_64
Current User     : kstbh
Home Directory   : /c/Users/kstbh
System Uptime    :
Current Date     : Sunday, 29 March 2026
------------------------------------------
License Note: This system and Git are both
covered under the GNU GPL v2 license.
==========================================
```

Explanation: It uses variables to store information such as the kernel version, current user, and system uptime, which are retrieved using command substitution ($()), showing how outputs of commands can be reused within a script. The use of echo statements helps in formatting the output neatly, making it more user-friendly instead of displaying raw terminal data

Script 2 — FOSS Package Inspector (Unit 2)

Write a script that checks whether your chosen software is installed on the system, finds its version, and uses a case statement to print a short description of its purpose based on the package name.

Concepts to use: if-then-else, case statement, rpm -qi or dpkg -l, pipe with grep.

Starter structure

```
#!/bin/bash
# Script 2: FOSS Package Inspector
PACKAGE="git"

echo "Checking for package: $PACKAGE..."

# Check if package is installed (using dpkg for Ubuntu/Debian)
if dpkg -l | grep -q "^ii  $PACKAGE "; then
    echo "SUCCESS: $PACKAGE is installed."
    echo "------------------------------------------"
    # Show Version and License info
    apt-cache show $PACKAGE | grep -E 'Version|Homepage'
    echo "License: GPL-2.0 (GNU General Public License v2)"
else
    echo "ERROR: $PACKAGE is NOT installed."
    echo "Please run: sudo apt install git"
fi

echo "------------------------------------------"
# Case statement for package philosophy
case $PACKAGE in
    git)
        echo "Git: The version control tool Linus built when proprietary failed
him." ;;
    mysql)
        echo "MySQL: Open source database at the heart of millions of apps." ;;
```

```
    apache2)
        echo "Apache: The web server that built the open internet." ;;
    *)
        echo "Software is the foundation of digital freedom." ;;
esac
```

```
$ bash "C:\Users\kstbh\OneDrive\Desktop\script2.sh"
Checking for package: git...
C:\Users\kstbh\OneDrive\Desktop\script2.sh: line 8: dpkg: command not found
ERROR: git is NOT installed.
Please run: sudo apt install git
-----------------------------------------
Git: The version control tool Linus built when proprietary failed him.
```

Explanation: This script checks whether a specific package like Git is installed on the system and provides useful details about it, acting like a simple audit tool. It uses an if-then-else structure to determine whether the package exists and prints appropriate messages based on the result, demonstrating decision-making in shell scripting. The use of pipes with grep helps filter out only relevant information such as version and license, avoiding unnecessary clutter.

Script 3 — Disk and Permission Auditor (Unit 2)

Write a script that loops through a list of important system directories and reports: how much space each uses, and the owner and permissions of the directory. Use a for loop.

Concepts to use: for loop, df, ls -ld, awk or cut to extract fields.

Starter structure

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
DIRS=("/etc" "/var/log" "/usr/bin" "/tmp")

echo "Starting Directory Audit..."
echo "-----------------------------------------"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        # Extract permissions, owner, and group using awk
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
        # Get size using du
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        echo "Directory: $DIR"
        echo "  - Permissions/Owner: $PERMS"
        echo "  - Total Size: $SIZE"
    else
        echo "Warning: $DIR does not exist."
    fi
done

echo "-----------------------------------------"
echo "Checking Git Configuration Audit..."
GIT_CONF="$HOME/.gitconfig"
```

```
if [ -f "$GIT_CONF" ]; then
    echo "Git Global Config Found at $GIT_CONF"
    ls -l "$GIT_CONF" | awk '{print "Permissions: " $1 " | Owner: " $3}'
else
    echo "Git global config not found. Run 'git config --global user.name' to create
one."
fi
```

```
$ bash "C:\Users\kstbh\OneDrive\Desktop\script3.sh"
Starting Directory Audit...
-----------------------------------------
Directory: /etc
  - Permissions/Owner: drwxr-xr-x kstbh 197609
  - Total Size: 1.6M
Warning: /var/log does not exist.
Directory: /usr/bin
  - Permissions/Owner: drwxr-xr-x kstbh 197609
  - Total Size: 91M
Directory: /tmp
  - Permissions/Owner: drwxr-xr-x kstbh 197609
  - Total Size: 866M
-----------------------------------------
Checking Git Configuration Audit...
Git Global Config Found at /c/Users/kstbh/.gitconfig
Permissions: -rw-r--r-- | Owner: kstbh
```

Explanation: It uses a for loop to iterate through a list of directories, demonstrating how repetitive tasks can be automated efficiently in shell scripting. Within the loop, conditional checks ensure that only existing directories are processed, while commands like ls -ld and du -sh are used to extract permissions and size information. Tools such as awk and cut help refine the output into a readable format.

Script 4 — Log File Analyzer (Units 2 & 5)

Write a script that reads a log file line by line, counts how many lines contain a keyword (like ERROR or WARNING), and prints a summary. Use a while-read loop and an if-then inside it.

Concepts to use: while read loop, if-then, counter variables, command-line arguments ($1).

Starter structure

```
#!/bin/bash
# Script 4: Log File Analyzer
LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0

# Check if file exists and is not empty
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi

# Do-while style check: If file is empty, wait or notify
if [ ! -s "$LOGFILE" ]; then
    echo "File is currently empty. Waiting for log entries..."
    sleep 2
fi

echo "Analyzing $LOGFILE for '$KEYWORD'..."

while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "----------------------------------------"
echo "Keyword '$KEYWORD' found $COUNT times."
echo "----------------------------------------"
echo "Last 5 matching entries:"
grep -i "$KEYWORD" "$LOGFILE" | tail -n 5
```

```
$ bash "C:\Users\kstbh\OneDrive\Desktop\script4.sh"
Error: File  not found.
```

Explanation: The main logic uses a while-read loop to go through the file line by line and an if condition to check for keyword matches, incrementing a counter each time a match is found. This combination of loops, conditions, and arithmetic operations illustrates how shell scripts can automate tasks that would otherwise require manual effort.

Script 5 — The Open-Source Manifesto Generator (Unit 5)

This is the most creative script. Write a script that generates a personalised open-source philosophy statement by asking the user three questions interactively, then composing a short paragraph using their answers and saving it to a .txt file.

Concepts to use: read for user input, string concatenation, writing to a file with >, date command, aliases concept demonstrated via a comment.

Starter structure

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator

echo "=== Personalized Open Source Manifesto ==="
read -p "1. Name one Git feature you use (e.g., branch, commit): " FEATURE
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. What would you share with the world for free? " SHARE

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

# Composing the manifesto
{
    echo "OPEN SOURCE MANIFESTO"
    echo "Created by $(whoami) on $DATE"
    echo "----------------------------------------"
    echo "I believe that software should be as open as a '$FEATURE'."
    echo "To me, freedom in code is defined by '$FREEDOM'."
    echo "In the spirit of Git, I commit to building '$SHARE'"
    echo "and sharing it freely with the global community."
} > "$OUTPUT"

echo "----------------------------------------"
echo "Manifesto saved to $OUTPUT"
echo "----------------------------------------"
cat "$OUTPUT"
```

```
$ bash "C:\Users\kstbh\OneDrive\Desktop\script5.sh"
=== Personalized Open Source Manifesto ===
1. Name one Git feature you use (e.g., branch, commit): commit
2. In one word, what does 'freedom' mean to you? Open
3. What would you share with the world for free? Open-source IDE
----------------------------------------
Manifesto saved to manifesto_kstbh.txt
----------------------------------------
OPEN SOURCE MANIFESTO
Created by kstbh on 29 March 2026
----------------------------------------
I believe that software should be as open as a 'commit'.
To me, freedom in code is defined by 'Open'.
In the spirit of Git, I commit to building 'Open-source IDE'
and sharing it freely with the global community.
```

It uses the read command to capture user input, demonstrating how shell scripts can interact with users rather than just executing commands. The script then combines these inputs with the current date using string concatenation to form a meaningful paragraph, which is saved to a text file using output redirection (>). Finally, it displays the saved content, ensuring immediate feedback. Beyond technical concepts like input handling and file writing, this script reflects the philosophical side of open source by encouraging users to think about their own contributions and values.